Project Vision, Requirements, and Design Document

Hotel Manager Enterprise Property Management System

Prepared By:	Kalel Roscoe, Shanterrian Walker, OBrien White, Jonathan Porter, KeiShaun Mortis, Silas Erving
Section & Group:	Group 3
Course Name & Number:	HCSC 460-01 (Senior Capstone)
Institution:	Morehouse College
Semester:	Fall 2026
Date Created:	November 11, 2026 (Version 1.0.0)

Revision History

Author(s)	Date	Reason For Changes	Version
Kalel Roscoe & Group 3	2026-09-15	Initial Proposal, Core Vision, Problem Statement	v0.1.0
Shanterrian Walker & OBrien White	2026-10-10	Room Inventory Domain Model & Alembic Baseline	v0.2.0
Jonathan Porter & KeiShaun Mortis	2026-10-27	RBAC, API Endpoints & Pytest Suite	v0.3.0
Silas Erving & Kalel Roscoe	2026-11-04	Resend Email Integration, Auth Layer & Guest Portal	v0.4.0
Group 3 (All Members)	2026-11-11	Pure PostgreSQL 16 Architecture & Final Specification	v1.0.0

1. Project Vision and Business Requirements

1.1 Background and Business Opportunity

Modern hospitality encompasses boutique hotels, bed-and-breakfasts, and regional properties. While multi-national chains utilize multi-million-dollar proprietary property management systems (PMS), independent operators struggle with costly legacy software or fragmented niche tools.

Heating, Ventilation, and Air Conditioning (HVAC) accounts for over 60% of commercial hotel energy expenditure. In traditional operations, electricity consumption is only discovered at the end of the billing cycle on a consolidated utility bill. Hotel Manager bridges this divide by delivering a cloud-native Enterprise PMS pairing core operations with real-time room electricity submetering telemetry.

1.2 Business Objectives and Success Criteria

- **Turnover Latency:** Reduce front-desk check-in/out times by at least 35%, completing state transitions in under 2 seconds.
- **Energy Overhead Savings:** Target a 15% to 22% reduction in unmonitored HVAC energy waste via room-level submeter telemetry.
- **Transactional Reliability:** Maintain 99.9% uptime backed by PostgreSQL 16 ACID transactions and foreign key constraints.
- **Automated Verification:** Maintain 100% test coverage across all services (49/49 automated unit and integration tests passing).

2. Vision and Scope of the Solution

2.1 Vision Statement

"For multi-property hospitality operators, facility engineers, front desk staff, and guests who require frictionless property administration and sustainable operations, Hotel Manager is an integrated, full-stack Property Management System. Unlike fragmented, expensive legacy platforms, Hotel Manager unites real-time room turnover, multi-hotel administrative provisioning, role-isolated operational consoles, and electrical submetering telemetry within a unified, high-performance web platform."

2.2 Major Features

1. **Multi-Property Provisioning:** Centralized management of hotels across multiple cities, states/provinces, and room counts.
2. **Authenticated RBAC:** PBKDF2-HMAC-SHA256 password hashing (100k rounds) with tamper-proof HMAC access tokens and 5 distinct roles.
3. **Rapid Front-Desk Room Turnover:** Real-time room status grid (available, OCCUPIED, Maintenance) with 1-click check-in/out.
4. **Room Inventory Multivariable Filtering:** Advanced room search by hotel, status, floor, room type, and nightly price bounds.
5. **Electricity Submeter Telemetry:** 1:1 room-to-meter association with in-app logging modal and quick-add increment buttons (+1 to +50 kWh).
6. **Facilities Maintenance Queue:** Work orders for rooms in maintenance with telemetry diagnostics and 1-click repair sign-off.
7. **Dedicated Guest Portal:** Self-service booking flow restricted to authenticated guest accounts with confirmation receipts (HM-XXXXXX).
8. **Resend Email Integration:** Transactional onboarding notifications and testing triggers via Resend SDK.

2.3 Thermodynamic & Energy Calculation Formulas

Sensible Heat Gain Formula:

Qs [BTU/hr] = 1.08 × CFM × ΔT

Where ΔT = Tf - T0 (°F indoor vs outdoor differential)

and CFM = (length × width × height) × 60 min/hr

Energy Cost Conversion:

Cost/hr = Qs [BTU/hr] × 0.000293071 [kW/(BTU/hr)] × Cost per kWh [$]

Cost/day = Cost/hr × 24 hr/day

3. Business Context & Schedule

System Stakeholders: Business Owners (Franchisees), System Administrators (ADMIN), Front Desk Staff (FRONT_DESK), Property Managers (PROPERTY_MANAGER), Maintenance Engineers (MAINTENANCE_ENGINEER), and Hotel Guests (GUEST). The project successfully advanced through Phase 1 (Architecture), Phase 2 (PostgreSQL Engine & Alembic), Phase 3 (React 19 & RBAC), Phase 4 (Resend Email & Guest Portal), and Phase 5 (Complete Verification & Pure PostgreSQL Documentation).

4. Preliminary and Detailed Design

4.1 Technology Stack

Tier	Component	Technology & Version	Purpose
Database	PostgreSQL	PostgreSQL 16	ACID relational persistence layer
Backend	FastAPI	FastAPI 0.115+	Asynchronous REST API framework
ORM	SQLAlchemy	SQLAlchemy 2.0+	ORM persistence and transactions
Frontend	React	React 19 & TypeScript	Role-based responsive SPA interface
Styling	Tailwind CSS	Tailwind CSS v4	Fluid percentage-based responsive layout
Email	Resend SDK	Resend 2.6+	Transactional email dispatches
Testing	Pytest	Pytest 9.1+ (49 tests)	100% automated test coverage

4.2 Database Design and Relational Schema

The relational database schema is structured across 5 primary tables in PostgreSQL 16: • **hotels:** id (PK), name, address, city, state (State/Province), country, contact_number, total_rooms, logo. • **rooms:** id (PK), hotel_id (FK), room_number, status (available/OCCUPIED/Maintenance), floor, nightly_price (NUMERIC), room_type. • **users:** id (PK), username (UNIQUE), email (UNIQUE), password_hash (PBKDF2), first_name, last_name, role (ENUM), other_info. • **hotel_users:** id (PK), hotel_id (FK), user_id (FK). • **electricity_meters:** id (PK), room_id (FK, UNIQUE), current_reading (FLOAT kWh), status (Active/Inactive).

5. Appendices & Verification

All 49 unit and integration tests execute successfully via uv run pytest. Frontend production build verified with npm run build (TypeScript static checking and Vite bundling). Full documentation available in repository under docs/PROJECT_VISION_REQUIREMENTS_DESIGN.md.